



Universidade Federal Fluminense
Curso: Ciência da Computação
Disciplina: Análise e Síntese de Algoritmos
Professor: Luís Felipe

Revisão P4

1. Dois algoritmos aproximativos A e B resolvem o mesmo problema de minimização. Para toda instância I , suas razões de aproximação são, respectivamente, $r_A = 1,5$ e $r_B = 2$.

- (a) Para uma instância I_1 , o algoritmo A retornou uma solução de valor 90. Sabendo que A tem razão de aproximação 1,5, determine os limites inferior e superior mais justos para $opt(I_1)$.

Resposta: Como o problema é de minimização e a razão de aproximação de A é 1,5, temos:

$$val(A(I_1)) \leq 1,5 \cdot opt(I_1) \Rightarrow opt(I_1) \geq \frac{90}{1,5} = 60.$$

Além disso, como $val(A(I_1)) = 90$ é viável, então $opt(I_1) \leq 90$. Logo, $60 \leq opt(I_1) \leq 90$.

- (b) Para uma instância I_2 , o algoritmo B retornou uma solução de valor 70. Sabendo que sua razão de aproximação é 2, determine os limites para $opt(I_2)$.

Resposta: A razão de aproximação de B garante que:

$$val(B(I_2)) \leq 2 \cdot opt(I_2) \Rightarrow opt(I_2) \geq \frac{70}{2} = 35.$$

Como $val(B(I_2)) = 70$ é viável, então $opt(I_2) \leq 70$. Logo, $35 \leq opt(I_2) \leq 70$.

- (c) Suponha que $opt(I_1) = opt(I_2)$. Qual algoritmo teve melhor desempenho relativo em sua instância?

Resposta: Sabendo que $opt(I_1) = opt(I_2)$, e usando os valores das soluções:

$$\text{Desempenho relativo de } A = \frac{val(A(I_1))}{opt(I_1)} = \frac{90}{opt}, \quad \text{de } B = \frac{val(B(I_2))}{opt(I_2)} = \frac{70}{opt}.$$

Como $\frac{90}{opt} > \frac{70}{opt}$, então o algoritmo B teve melhor desempenho relativo na sua instância, apesar de sua razão de aproximação teórica ser pior.

- (d) É possível que $opt(I_1) = 65$? Justifique com base na razão de aproximação de A e o valor retornado.

Resposta: Se $opt(I_1) = 65$, então:

$$val(A(I_1)) = 90 \leq 1,5 \cdot 65 = 97,5,$$

o que satisfaz a definição de razão de aproximação. Além disso, $opt(I_1) = 65$ pertence ao intervalo $[60, 90]$ encontrado na letra (a), portanto sim, é possível.

2. Considerando o problema $3SAT$ onde cada cláusula possui exatamente três literais, responda abaixo:

- (a) Fixada uma cláusula C qualquer, qual a probabilidade de satisfazermos C se escolhermos uma atribuição de suas variáveis aleatoriamente?

Resposta: Seja E uma expressão booleana com n variáveis $x_1, \dots, x_n$ e m cláusulas, cada uma contendo três variáveis. Seja $C = (y_i \vee y_j \vee y_k)$ uma cláusula qualquer, em que $y_l \in \{x_l, \overline{x_l}\}$ para $l \in \{i, j, k\}$. Para que C não seja satisfeita por uma atribuição das variáveis $x_1, \dots, x_n$, devemos ter $y_i = y_j = y_k = F$, enquanto as demais variáveis não relacionadas a C podem receber tanto F quanto V . Com isso, as variáveis associadas a C são fixas e as demais cada uma pode ter duas possibilidades de valores. Assim, o número de atribuições as variáveis que não satisfazem C é 2^{n-3} . Como há no total 2^n atribuições para todas as variáveis, pela complementaridade, temos que o número de atribuições que satisfazem C é $2^n - 2^{n-3} = 2^n(1 - \frac{1}{8}) = \frac{7}{8}2^n$. Portanto, a probabilidade desejada é $\frac{\frac{7}{8}2^n}{2^n} = \frac{7}{8}$.

- (b) Mostre que cada instância de $3SAT$ existe uma atribuição de suas variáveis que satisfaz pelo menos $\frac{7}{8}$ das cláusulas.

Resposta: Seja E a expressão FNC com n variáveis e m cláusulas $C_1, \dots, C_m$, cada uma com três literais. Seja X_i a variável indicadora tal que $X_i = 1$ se C_i é satisfeita, e $X_i = 0$ se C_i não é satisfeita. Como X_i é uma variável indicadora, temos que o valor esperado de X_i é precisamente sua probabilidade de sucesso, que pela questão a) vale $\frac{7}{8}$. Ou seja, $E(X_i) = \frac{7}{8}$. Assim, o número de cláusulas satisfeitas é $X = \sum_{i=1}^m X_i$. Pela linearidade da esperança, o valor esperado de X é $E(X) = E(\sum_{i=1}^m X_i) = \sum_{i=1}^m E(X_i) = \sum_{i=1}^m \frac{7}{8} = \frac{7}{8}m$. Como o valor esperado é $\frac{7}{8}m$ então há uma atribuição que satisfaz pelo menos $\frac{7}{8}m$, como segue da consequência do valor esperado.

3. Dado um grafo, considere a seguinte estratégia:

- 1) se existir vértice no grafo, tome um vértice de grau mínimo e adicione-o ao conjunto F , inicialmente vazio;
- 2) remova seus vizinhos e volte para o passo anterior.

- (a) Ao final deste algoritmo, F retornará um conjunto de vértices que possui qual propriedade?

Resposta: Conjunto independente maximal.

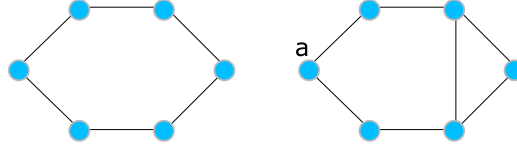


Figura 1: Figuras da Questão 3c. O exemplo da esquerda, a solução gulosa retorna a resposta ótima, enquanto o da direita, se escolhermos o vértice x na primeira etapa então a solução do algoritmo não será ótima.

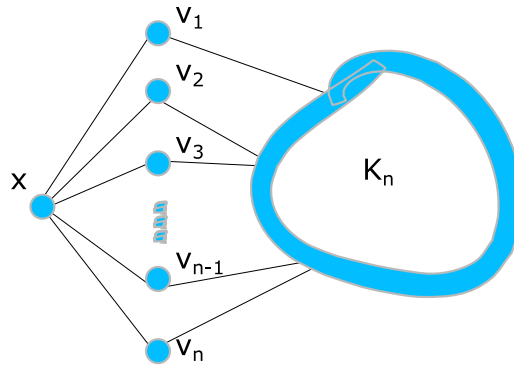


Figura 2: Figura da Questão 3c que mostra um caso que o algoritmo guloso nunca obtém a solução ótima e esse erro é em função linear do tamanho da entrada.

- (b) Para qual problema de otimização podemos aplicar este algoritmo? Este problema é de minimização ou maximização?

Resposta: Maximização.

- (c) Exiba um exemplo de grafo para o qual o algoritmo retorne a solução ótima para o problema, **E** um outro exemplo para o qual o algoritmo não retorne a solução ótima para o problema.

Resposta: Considere a Figura 1. Na figura da esquerda o algoritmo retorna o conjunto independente máximo, enquanto o da direita, ao tomar o vértice a , a solução apresentada tem tamanho 2, enquanto que a solução ótima tem tamanho 3.

Um exemplo mais geral pode ser visto na Figura 2. Nele, o vértice x é vizinho a cada vértice v_i , $i = 1, \dots, n$. Os vértices $v_1, \dots, v_n$ formam um conjunto independente, e cada um desses vértices v_i é vizinho a todos os vértices do conjunto K_n . Esse último é um subgrafo completo com n vértices. O vértice x é o de menor grau no grafo, pois possui grau n , enquanto cada vértice v_i possui grau $n + 1$ e os vértices de K_n possuem grau $n - 1 + n = 2n - 1$. Dessa forma, a solução gulosa retornará resposta 2, com o vértice x e um vértice de K_n . Porém, a solução ótima tem tamanho n , com todos os vértices v_i , para $i = 1, \dots, n$.

- (d) Prove que a razão de aproximação deste algoritmo é $\frac{1}{\Delta+1}$, onde Δ é o grau máximo do grafo.

Resposta: Primeiramente, vamos argumentar que o algoritmo guloso retorna um conjunto independente S de tamanho $|S| \geq \frac{n}{\Delta+1}$, onde Δ é o grau máximo do grafo, ou

seja, Δ o maior grau dentre todos os vértices do grafo.

Considere o conjunto $V \setminus S$, vamos apresentar um limite superior do número de vértices nesse conjunto da seguinte forma: Um vértice u pertence a $V \setminus S$ quando u é removido do grafo pelo fato dele ser vizinho a algum vértice $v \in S$ e v é adicionado a S pelo algoritmo guloso. Como cada vértice v em S possui no máximo Δ vizinhos, então $|V \setminus S| \leq \Delta|S|$. Ou seja, o número de vértices fora de S é limitado superiormente por cada vértice de S ter grau Δ . Como cada vértice exclusivamente pertence a S ou pertence a $V \setminus S$, temos que $|S| + |V \setminus S| = n$, e portanto: $|S| + \Delta|S| \geq n$, o que implica $|S| \geq \frac{n}{\Delta+1}$.

Como um limite superior para o tamanho de S é n , temos então a razão $\frac{\frac{n}{\Delta+1}}{n} = \frac{1}{\Delta+1}$.

4. Certo exame de sangue pode ser entendido como um algoritmo de Monte Carlo baseado-no-sim que, com probabilidade $p \geq 95\%$, diagnostica determinada doença X caso o dono do sangue examinado de fato a possua. Uma epidemia de X fez com que um terço dos habitantes de uma cidade estivesse com aquela doença, cujo tratamento é, no entanto, muito penoso e não deve ser administrado a pessoas sãs. Quantas vezes aquele exame precisará ser repetido até possa ser avaliada como “desprezível” (menor do que 1%) a chance de que uma pessoa daquela cidade apresente a doença X ?

Resposta: Considere os seguintes eventos associados a uma execução do algoritmo que consiste na aplicação do exame de sangue em um paciente:

- A_S : o algoritmo responde SIM
- A_N : o algoritmo responde NÃO
- C_S : a resposta correta para aquela entrada é SIM
- C_N : a resposta correta para aquela entrada é NÃO

Pelos dados do enunciado, temos que:

- $P(A_S|C_S) = 0,95$: Se a pessoa realmente está doente, o exame responde SIM com 95% de certeza.
- $P(A_N|C_S) = 0,05$: Existe 5% de chance de erro quando a pessoa está doente (ou seja, o exame não indica doença).
- $P(A_S|C_N) = 0$: Como o algoritmo (exame) é baseado no SIM, não há erro quando ele indica doença.
- $P(A_N|C_N) = 1$: Como o algoritmo (exame) é baseado no SIM, não há possibilidade de erro quando o paciente não está doente (ou seja, não há falso positivo).

Observe ainda, que nos são informados os valores $P(C_S) = 1/3$ e $P(C_N) = 2/3$.

Suponha que nosso algoritmo consiste não apenas em aplicar o exame uma vez, mas em repeti-lo k vezes. Neste caso, $P(A_N|C_S) = 0,05^k$ e $P(A_S|C_S) = 1 - 0,05^k$.

Nosso objetivo é descobrir, dado que o exame não indicou doença, qual a real probabilidade de o paciente estar doente (queremos que este valor seja menor que 1%). Ou seja, queremos saber $P(C_S|A_N)$. Temos que:

$$P(C_S|A_N) = \frac{P(C_S, A_N)}{P(A_N)} = \frac{P(A_N|C_S)P(C_S)}{P(A_N|C_S)P(C_S) + P(A_N|C_N)P(C_N)}$$

Substituindo pelos valores conhecidos,

$$P(C_S|A_N) = \frac{(0,05)^k(1/3)}{(0,05)^k(1/3) + 2/3}$$

Basta agora descobrir para que valores de k a expressão acima é menor do que 1%. Efetuando as contas, vemos que, para uma única repetição do exame, a chance de “falso negativo” é de 2,45%. Se efetuarmos dois exames, essa chance já será de 0,12%, atingindo nosso objetivo com apenas duas repetições.

5. Seja o mesmo exame de sangue do exercício 4. Não há epidemia alguma, desta vez. Um médico experiente, porém baseado nos diversos sintomas clínicos apresentados por um paciente seu, avalia em 80% a probabilidade de que aquele paciente tenha a doença X. O exame que se segue, no entanto, não revela a existência da doença. Como aquele médico deve reavaliar sua confiança inicial de que seu paciente é um doente de X?

Resposta: O médico avaliou seu paciente e indicou que com 80% de chances, seu paciente tem a doença, então $P(C_S) = 0,8$.

O exame indicou que o paciente não tem a doença. Então a probabilidade do exame dar errado, dado que o paciente tem a doença é $P(A_N|C_S) = 1 - p = 1 - 0,95 = 0,05$. Onde p é o mesmo do item anterior.

A pergunta feita é como o médico garantir que sua avaliação esteja certa. Ou seja, que de fato o paciente tenha a doença, dado que o exame deu negativo para a doença. Isso quer dizer que queremos calcular $P(C_S|A_N)$.

Assim, por enquanto a gente possui $P(A_N|C_S)$ e $P(C_S)$, e para obter o que queremos, precisamos então utilizar a Regra de Bayes. Para isso, precisamos ter mais algumas informações, que conseguimos do item anterior, como: $P(A_N|C_N) = 1$ (o exame não apresenta falso positivo, pois é baseado-no-sim). $P(C_N) = 1 - P(C_S) = 0,2$ (complemento de 80% que foi dado no enunciado)

Agora só aplicar a Regra de Bayes:

$$P(C_S|A_N) = \frac{P(A_N|C_S).P(C_S)}{P(A_N)}$$

Além disso, $P(A_N) = P(A_N|C_S).P(C_S) + P(A_N|C_N).P(C_N)$.

6. Considere o problema onde deseja-se buscar um elemento em uma lista com repetições que possui as seguintes restrições:

- Ao obter o primeiro elemento que é desejado o algoritmo deve encerrar.
- A lista contém n elementos, e cada um possui valor dentre 1, 2, 3, 4, 5.
- Todos os cinco elementos são distribuídos em igual quantidade.
- Nada se sabe sobre a localização dos elementos.

Desenvolva um algoritmo randomizado para obter um elemento $x \in \{1, 2, 3, 4, 5\}$ na lista. Qual tipo de algoritmo randomizado é este? Além disso, determine e mostre o valor esperado da sua complexidade.

Resposta: Exemplo similar feito em aula, faça da mesma maneira.

7. Em grafos, um PROBLEMA SANDUÍCHE PARA Π é definido da seguinte forma: dados dois grafos $G^1 = (V, E_1)$ e $G^2 = (V, E_2)$, queremos decidir se existe um grafo $G = (V, E)$ tal que $E_1 \subseteq E \subseteq E_2$ e que satisfaça a propriedade Π . Se este grafo existir, ele é dito **grafo sanduíche** para (G^1, G^2) . Para muitas classes de grafos, este problema é NP-completo. Por exemplo, saber se é possível obter um grafo sanduíche cordal para uma instância (G^1, G^2) é NP-completo.

Uma **restrição** do PROBLEMA SANDUÍCHE PARA Π é o PROBLEMA PROBE PARTICIONADO PARA Π , definido da seguinte maneira: dados um grafo $G^1 = (V, E_1)$ e uma partição (P, N) de V , onde N é um **conjunto independente**, deseja-se saber se é possível obter um grafo $G = (V, E_2)$ de modo que $E_1 \subseteq E_2$ e satisfazendo Π , APENAS pela adição de arestas entre vértices de N .

Responda:

Sabe-se que o PROBLEMA PROBE PARTICIONADO PARA GRAFOS CORDAIS é solucionável em tempo polinomial. Apresente um algoritmo randomizado para solucionar o PROBLEMA SANDUÍCHE PARA GRAFOS CORDAIS, que é NP-completo, utilizando o problema probe particionado para grafos cordais. (Note que não é necessário conhecimento da classe dos grafos cordais para solucionar esta questão.) Que tipo de algoritmo é esse? Justifique.

Resposta: O PROBLEMA SANDUÍCHE PARA GRAFOS CORDAIS é NP-completo, mas o PROBLEMA PROBE PARTICIONADO PARA GRAFOS CORDAIS pode ser resolvido em tempo polinomial. A seguir, apresentamos um algoritmo randomizado que utiliza o problema particionado para abordar o problema sanduíche. Este algoritmo é classificado como um algoritmo de Monte Carlo, pois utiliza aleatoriedade e possui uma probabilidade controlada de erro.

O objetivo é determinar se existe um grafo $G = (V, E)$ tal que $E_1 \subseteq E \subseteq E_2$ e G seja cordal. O algoritmo funciona da seguinte maneira: inicialmente, seja (G^1, G^2) uma instância do problema, onde $G^1 = (V, E_1)$ e $G^2 = (V, E_2)$. Realiza-se uma escolha aleatória para particionar o conjunto de vértices V em dois subconjuntos disjuntos (P, N) , onde N é garantido como um conjunto independente (sem arestas entre seus vértices em E_1).

Após a partição, resolve-se o PROBLEMA PROBE PARTICIONADO PARA GRAFOS CORDAIS utilizando o grafo G^1 e a partição (P, N) , adicionando arestas apenas entre vértices de N . Caso o grafo resultante $G = (V, E)$ satisfaça as condições $E_1 \subseteq E \subseteq E_2$ e seja cordal, retornamos G como uma solução válida. Caso contrário, o processo é repetido com uma nova escolha aleatória da partição (P, N) , até que uma solução seja encontrada ou até atingir um número máximo de iterações.

Esse algoritmo é um algoritmo de Monte Carlo porque utiliza escolhas aleatórias na definição da partição (P, N) e pode não garantir uma solução válida em todas as execuções. Ele é baseado no "sim", pois assume inicialmente que um grafo cordal sanduíche pode existir e trabalha tentando construí-lo. Caso uma tentativa falhe, o algoritmo não conclui que a

solução não existe, mas repete o processo explorando outras configurações. Além disso, a probabilidade de erro pode ser controlada pelo número de iterações: com mais tentativas, aumenta-se a chance de encontrar uma solução válida, se ela existir.

Concluimos que este algoritmo randomizado utiliza o PROBLEMA PROBE PARTICIONADO PARA GRAFOS CORDAIS como sub-rotina para abordar o PROBLEMA SANDUÍCHE PARA GRAFOS CORDAIS. Sua classificação como algoritmo de Monte Carlo reflete o uso de aleatoriedade e a possibilidade de erro controlado. Ele é baseado no "sim", pois busca ativamente construir uma solução ao invés de provar sua inexistência.

8. Seja um algoritmo randomizado de verificação de primalidade com erro apenas nos compostos (ou seja, um algoritmo de Monte Carlo baseado no não). Suponha que, para cada inteiro composto n , o algoritmo pode retornar incorretamente “primo” com probabilidade no máximo $1/4$.

- (a) Quantas execuções independentes são necessárias para que a probabilidade de erro seja menor que $1/1000$?

Resposta: Como o erro ocorre apenas quando o número é composto e o algoritmo responde “primo”, a probabilidade de erro em uma execução é no máximo $1/4$. Se o algoritmo for executado k vezes independentemente e responder “primo” em todas, a chance de erro (ou seja, de o número ser composto) é no máximo:

$$\left(\frac{1}{4}\right)^k.$$

Queremos que essa probabilidade seja inferior a $1/1000$:

$$\left(\frac{1}{4}\right)^k < \frac{1}{1000}.$$

Tomando logaritmos:

$$k > \frac{\log(1/1000)}{\log(1/4)} \approx \frac{-3}{-0,60206} \approx 4,98.$$

Portanto, com $k = 5$ execuções, a probabilidade de erro será inferior a $0,1\%$.

- (b) Suponha que o algoritmo tenha sido executado 5 vezes consecutivas retornando “primo”. Qual a probabilidade de erro nessa sequência de testes?

Resposta: Se o número testado for composto, a chance de o algoritmo falhar (retornar “primo”) em todas as 5 execuções é no máximo:

$$\left(\frac{1}{4}\right)^5 = \frac{1}{1024} \approx 0,00098 \approx 0,098\%.$$

Assim, a chance de que esse número seja composto mesmo após 5 respostas “primo” é inferior a $0,1\%$.

- (c) Qual o tipo de erro que esse algoritmo pode cometer? Há falsos negativos?

Resposta: O algoritmo é de Monte Carlo baseado no “não”, isto é:

- Se o número for composto, o algoritmo pode erroneamente responder “primo” (erro).

- Se o número for primo, o algoritmo sempre responde corretamente “primo” (sem erro).
- Se o algoritmo responde “composto”, a resposta está sempre correta.

Portanto, o único tipo de erro possível é o falso positivo (responder “primo” para um número composto). Não há falsos negativos.

9. Um algoritmo de Monte Carlo baseado no “não” é utilizado para testar se um número n é primo. Esse algoritmo pode errar apenas ao dizer “primo” para um número composto, com probabilidade de erro em uma execução dada por p ($0 < p < 1$).

Considere agora uma estratégia adaptativa: o algoritmo é executado repetidamente sobre a mesma entrada até que a resposta “composto” seja obtida, ou até atingir um limite de k repetições.

- (a) Qual a probabilidade de o algoritmo parar antes do limite k ao testar um número composto?

Resposta: Se o número n for composto, o algoritmo pode detectar isso corretamente com probabilidade $1 - p$ em cada execução. A probabilidade de ele parar antes da k -ésima execução é a probabilidade de que pelo menos uma execução retorne “composto” entre as k tentativas. Isso corresponde ao complemento da probabilidade de ele falhar (responder “primo”) em todas as k execuções:

$$P(\text{parar antes de } k) = 1 - p^k.$$

- (b) Se o número for composto e o algoritmo executar k vezes sem parar (ou seja, respondeu “primo” em todas), qual a chance de erro?

Resposta: Neste caso, o algoritmo retornou “primo” em todas as k execuções, o que só é possível se todas falharam. A probabilidade de isso ocorrer (dado que o número é composto) é:

$$P(\text{erro após } k \text{ execuções}) = p^k.$$

Portanto, a chance de erro final é p^k .

- (c) Esse método aumenta ou reduz a taxa de erro final? Justifique.

Resposta: Esse método reduz a taxa de erro final em comparação com uma única execução. Na abordagem original (1 execução), a chance de erro é p . Ao repetir até k vezes, a chance de errar em todas e, portanto, de sair com a resposta errada, passa a ser p^k , que é muito menor que p para $k \geq 2$. Ou seja, a estratégia adaptativa reduz significativamente a chance de falso positivo, embora aumente o tempo de execução esperado.

10. Um algoritmo de Monte Carlo é usado para identificar corretamente se um número possui uma certa propriedade (por exemplo, ser primo). Cada execução

retorna a resposta correta com probabilidade $p = 0,95$, independentemente das anteriores.

Suponha que o algoritmo seja executado $k = 10$ vezes e que o resultado final será decidido por maioria das respostas (isto é, o valor que aparecer mais vezes será aceito como resposta).

- (a) Modele o número de respostas corretas como uma variável aleatória. Qual a distribuição?

Resposta: Seja X a variável aleatória que conta o número de execuções com resposta correta. Temos $X \sim \text{Bin}(k = 10, p = 0,95)$.

- (b) Qual a probabilidade de o algoritmo retornar a resposta correta por maioria (ou seja, obter pelo menos 6 respostas corretas)?

Resposta: Queremos $P(X \geq 6)$, onde $X \sim \text{Bin}(10, 0,95)$. Como p é alto, a chance de maioria correta é bem alta. Podemos calcular:

$$P(X \geq 6) = 1 - P(X \leq 5) = 1 - \sum_{i=0}^5 \binom{10}{i} (0,95)^i (0,05)^{10-i}.$$

Esse valor pode ser aproximado numericamente como:

$$P(X \geq 6) \approx 1 - 3,17 \times 10^{-6} \approx 0,9999968.$$

Ou seja, há mais de 99,999% de chance de maioria correta com 10 execuções.

- (c) Qual é o valor esperado do número de respostas corretas?

Resposta: Se $X \sim \text{Bin}(10, 0,95)$, então:

$$\mathbb{E}[X] = np = 10 \cdot 0,95 = 9,5.$$

- (d) Esse algoritmo é de tipo Las Vegas ou Monte Carlo? Justifique.

Resposta: O algoritmo é de Monte Carlo, pois:

- Sempre termina em tempo fixo (10 execuções),
- Mas pode retornar uma resposta incorreta (se a maioria for incorreta, o que é raro mas possível).

Portanto, ele tem tempo determinístico e erro probabilístico, características típicas de Monte Carlo.

11. Suponha que desejamos gerar uma permutação aleatória de um vetor de n elementos usando o seguinte algoritmo de Las Vegas:

- Para cada posição i do vetor (de 1 a n), sorteie uma posição aleatória j entre i e n e troque $A[i]$ com $A[j]$.
- Suponha que a função de sorteio (`random(j)`) pode falhar com probabilidade p — ou seja, ela retorna uma posição fora do intervalo válido (um erro detectável, que força repetir a tentativa).

- O algoritmo repete o sorteio até obter uma posição válida. A cada chamada, registra-se se foi bem-sucedida ou não.

Considere agora o processo de execução total da função `random(j)` durante a construção da permutação.

- (a) Para um valor fixo de n e para cada posição i , quantas vezes em média o algoritmo precisa chamar `random(j)` até obter um valor válido?

Resposta: Cada tentativa tem sucesso com probabilidade $1 - p$, então o número de tentativas até o primeiro sucesso segue uma variável geométrica com parâmetro $1 - p$. Logo, o número esperado de tentativas é:

$$\mathbb{E}[X] = \frac{1}{1 - p}.$$

- (b) Seja T o número total de chamadas `random(j)` feitas ao longo da execução completa do algoritmo (em todas as n posições). Modele o número de chamadas bem-sucedidas como uma variável aleatória binomial. Qual a distribuição?

Resposta: Cada chamada `random(j)` tem probabilidade $1 - p$ de sucesso (valor válido). Se o total de chamadas feitas é T , e consideramos fixo esse número, então o número de chamadas bem-sucedidas é uma variável aleatória binomial:

$$X \sim \text{Bin}(T, 1 - p).$$

- (c) Se o algoritmo precisa de exatamente n sucessos para terminar (uma para cada posição), qual o valor esperado de chamadas totais T ?

Resposta: Para cada uma das n posições, o número de chamadas até obter um sucesso é geométrica com parâmetro $1 - p$.

Como a soma de n variáveis geométricas com parâmetro $1 - p$ tem valor esperado:

$$\mathbb{E}[T] = \sum_{i=1}^n \mathbb{E}[X_i] = n \cdot \frac{1}{1 - p}.$$

Portanto, em média, o algoritmo fará $\frac{n}{1-p}$ chamadas para concluir.

- (d) O algoritmo é do tipo Las Vegas ou Monte Carlo? Justifique.

Resposta: O algoritmo é do tipo Las Vegas, pois:

- Sempre retorna uma permutação correta e uniforme (resultado garantido),
- O número de chamadas à função `random` é aleatório, dependendo dos erros.

Portanto, ele é correto com probabilidade 1, mas tem tempo de execução variável.